

Trabajo Práctico 3 — Un personaje que se mueve

Diplomatura de Videojuegos · Clase 3 Objetivo: hacer un **personaje controlable** de verdad. Vas a crear un `CharacterBody2D`, moverlo en **4 direcciones** con **acciones del Input Map**, usar `delta` para que sea parejo en cualquier máquina, **correr** (con `just_pressed` / `just_released`), hacer un **dash** (con `just_pressed`) y **limitar la pantalla** con `clamp()`.

Qué vas a lograr

Un personaje en pantalla que:

- se **mueve en 4 direcciones** con el teclado (acciones que **vos** vas a mapear),
- se mueve **igual de rápido** en cualquier compu, gracias a `delta`,
- **corre** más rápido mientras mantenés una tecla, y vuelve a caminar al soltarla,
- hace un **dash** (impulso) que se dispara **una sola vez** por pulsación,
- **no puede salir** de la pantalla.

 **Tiempo estimado:** 60–90 min. Como siempre: **escribí el código vos**. Se aprende tecleando y probando.

 **Viene de la Clase 3:** hoy juntamos todo lo del *game loop* — `_ready()` para preparar, `_process(delta)` para cada frame, y la **regla de oro**: *todo movimiento se multiplica por `delta`*.

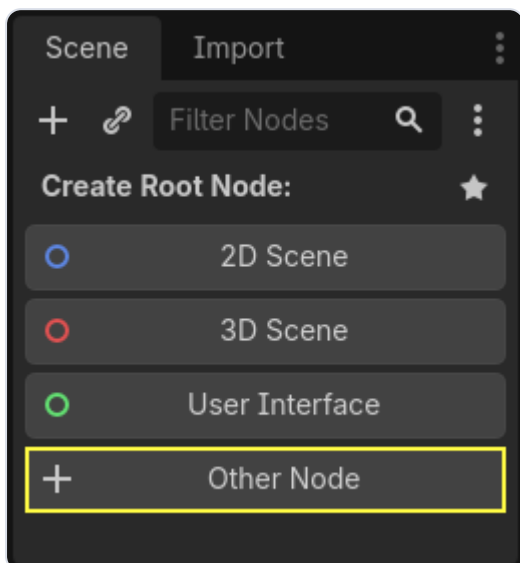
Cómo va a quedar el árbol de nodos

Jugador (<code>CharacterBody2D</code>)	← el personaje controlable
└ Sprite2D	← su imagen
└ CollisionShape2D	← su cuerpo físico

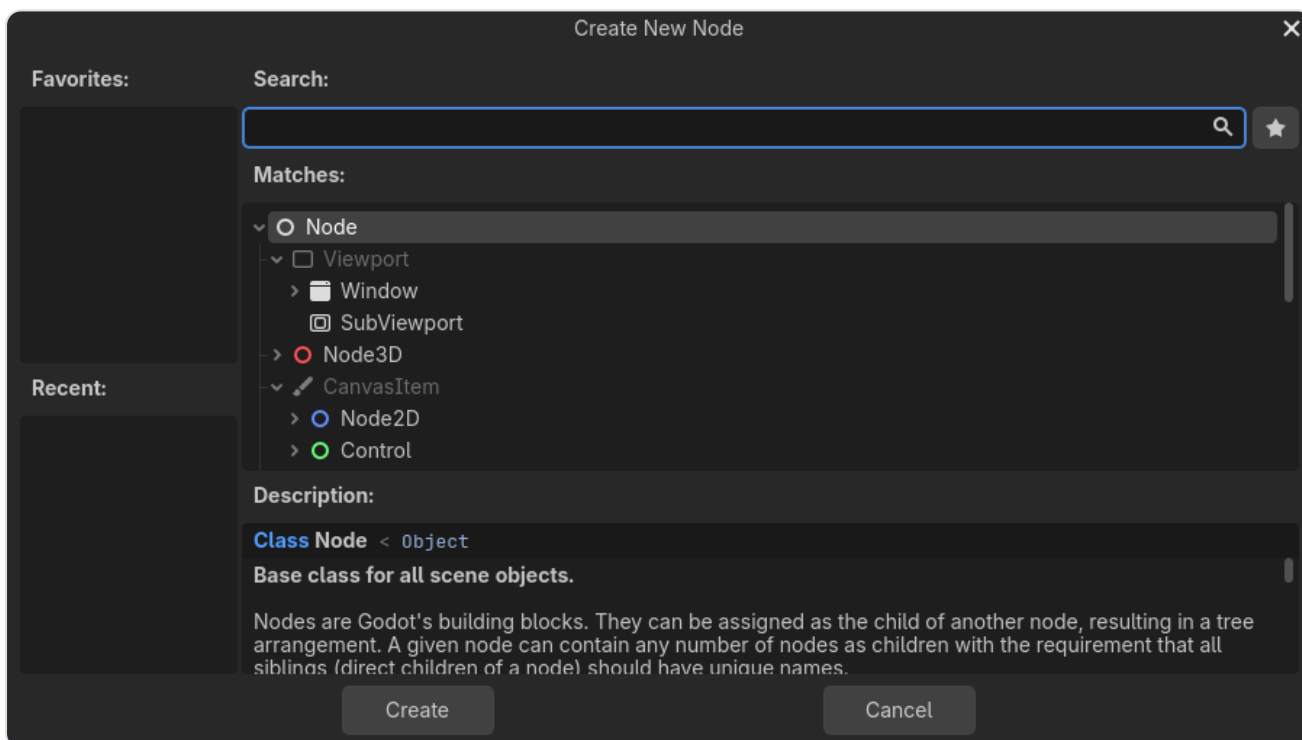
Un solo personaje, con su imagen y su forma. El script va **en el nodo raíz** `Jugador`.

🔧 Parte 0 — Crear el personaje (CharacterBody2D)

1. Abrió Godot → **New Project** → nombre `tp3-movimiento` → carpeta vacía → **Create & Edit**.
2. En el panel **Escena**, clic en **Otro Nodo** (*Other Node*).



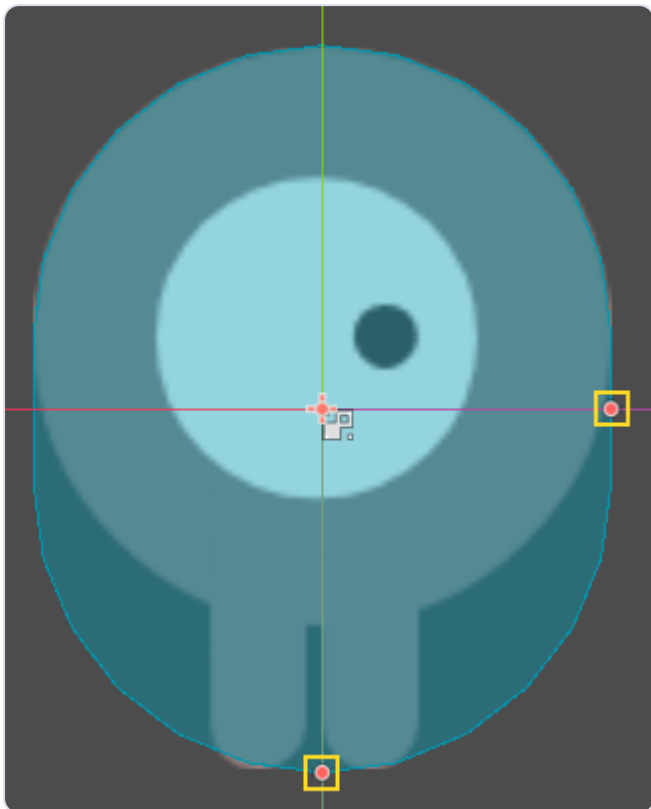
3. Buscá `CharacterBody2D`, selecciónalo y **Crear**. Renombralo a `Jugador`.



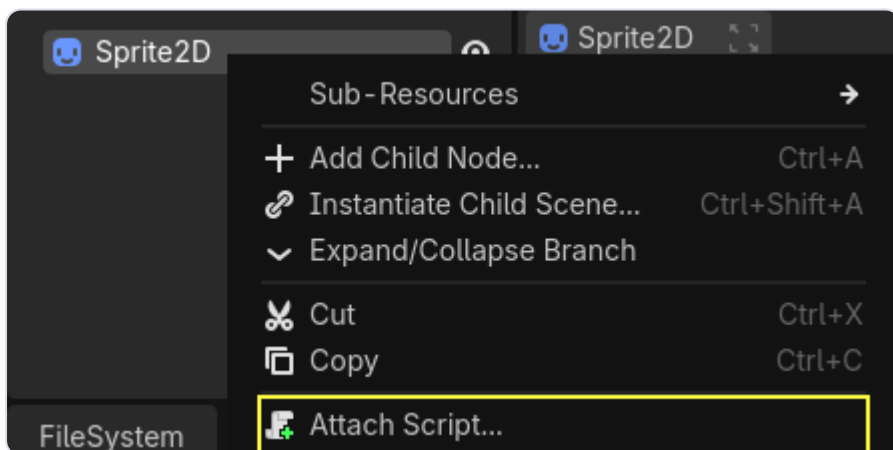
💡 **¿Por qué `CharacterBody2D` ?** Es el nodo pensado para **personajes controlados por el jugador** (lo conocimos en la Clase 1). Por ahora lo movemos con `position` (como en clase); en la **clase de física** vamos a usar su sistema de `velocity` + `move_and_slide()` para que además **choque** con las paredes.

4. **Darle imagen:** con `Jugador` seleccionado, agregá un hijo (**Ctrl + A**) → `Sprite2D`. Arrastrá el `icon.svg` del panel **FileSystem** hasta la propiedad **Texture** del Inspector.

5. **Darle cuerpo:** seleccioná `Jugador` de nuevo → hijo (**Ctrl + A**) → `CollisionShape2D`. En el Inspector → **Shape** → **Nuevo RectangleShape2D** y ajustá los puntos naranjas para cubrir el sprite.



6. **Adjuntar el script:** clic derecho sobre `Jugador` → **Attach Script** → dejá `res://jugador.gd` → **Create**.



7. Borrá la plantilla y dejá esto:

```
extends CharacterBody2D

func _ready():
    print("=== Personaje listo ===")
```

8. Guardá (**Ctrl + S** → `jugador.tscn`) y ejecutá con **F6**. En el panel **Output** tiene que aparecer el mensaje, y en la ventana el ícono de Godot.

```
Godot Engine v4.6.rc1.official
D3D12 12_0 - Forward+ - Using
1060 6GB

Hello, world!
--- Debugging process stopped
```

✅ **Punto de control 0:** ves el personaje en la ventana y el mensaje en Output.

Parte 1 — Variables y `_ready()`

Concepto: crear variables y mostrarlas por consola en `_ready()`.

Reemplazá el script por esto y apretá **F6**:

```
extends CharacterBody2D

# --- Configuración del personaje ---
var nombre = "Explorador"
var velocidad_normal = 200      # píxeles por segundo
var velocidad_rapida = 400      # cuando corre
var velocidad = velocidad_normal # la que se usa ahora

func _ready():
    print("=== " + nombre + " listo ===")
    print("Velocidad normal: " + str(velocidad_normal) + " px/seg")
    print("Velocidad al correr: " + str(velocidad_rapida) + " px/seg")
```

Deberías ver en Output:

```
=== Explorador listo ===
Velocidad normal: 200 px/seg
Velocidad al correr: 400 px/seg
```

 `_ready()` corre **una sola vez**, al aparecer el nodo. Es el lugar para **preparar el estado inicial** e imprimir la configuración. Fijate que `velocidad` **arranca** valiendo `velocidad_normal`: más adelante la vamos a cambiar cuando el personaje corra.

✅ **Punto de control 1:** imprimís la config del personaje al arrancar.

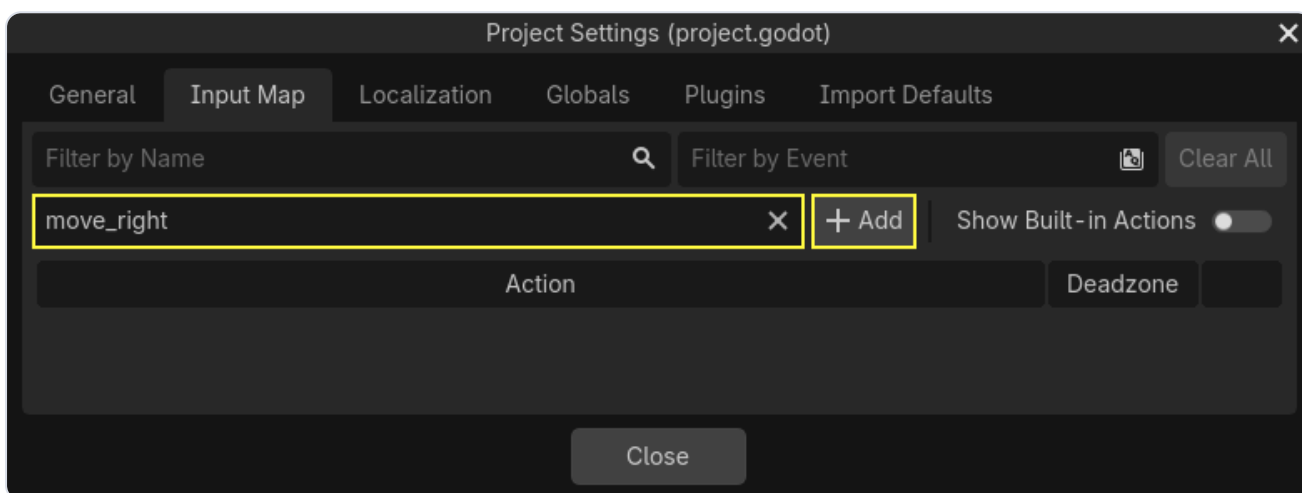
🎮 Parte 2 — El Input Map: acciones con nombre

Concepto: movimiento en 4 direcciones con **mapeo de entrada** (Input Map).

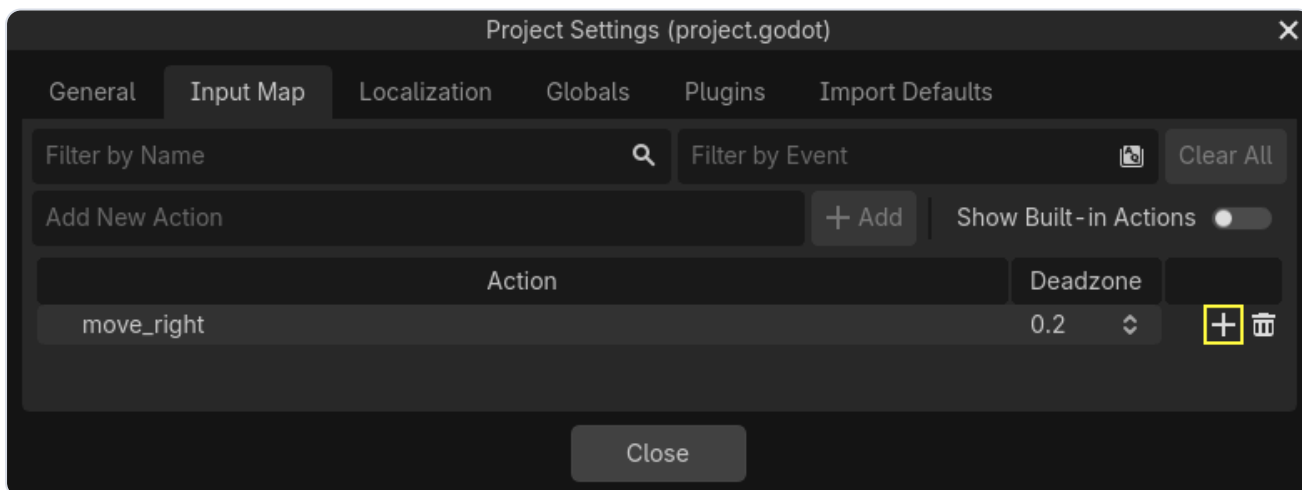
En vez de escribir la tecla en el código (frágil), creamos **acciones con nombre**. Si mañana querés cambiar la tecla, la cambiás **en un solo lugar**.

Abrí **Project** → **Project Settings** → **Input Map** (*Proyecto* → *Configuración del proyecto* → *Mapa de Entrada*).

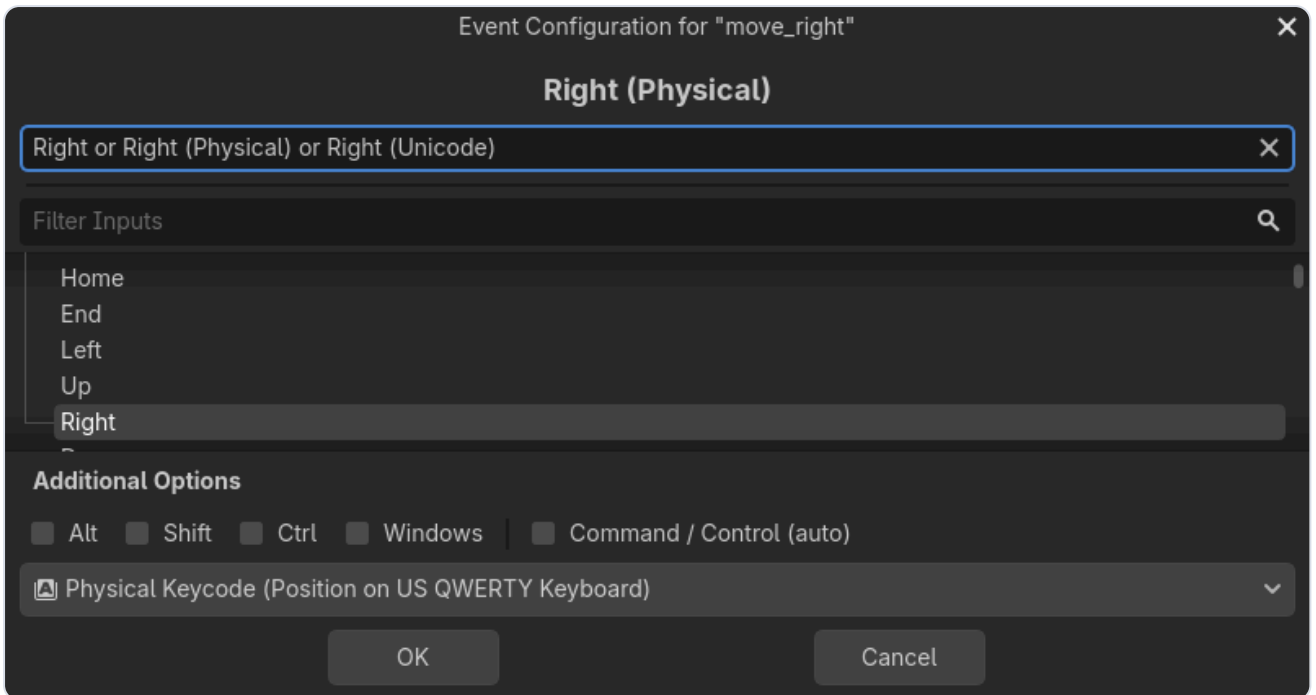
1. Escribí el nombre de la acción, `mover_derecha`, en la barra de arriba y clic en **Add** (*Añadir*).



2. En la acción recién creada, clic en el **+** de la derecha para asignarle una tecla.

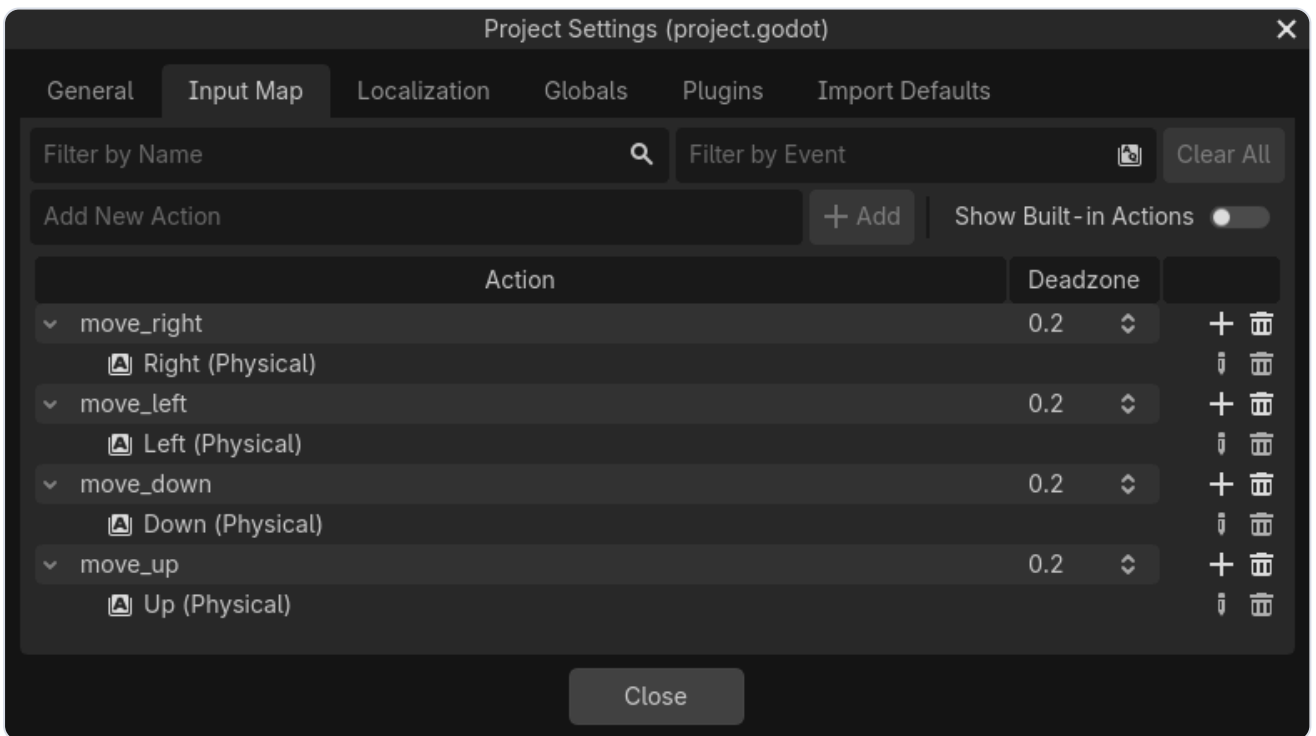


3. **Presioná la tecla** que querés (por ejemplo **D** o la flecha →) y confirmá con **OK**.



4. Repetí para las otras tres direcciones. Te tiene que quedar así:

ACCIÓN	TECLA SUGERIDA
mover_derecha	D o →
mover_izquierda	A o ←
mover_arriba	W o ↑
mover_abajo	S o ↓



📸 En la captura oficial las acciones se llaman `move_right` , `move_left` ... Nosotros usamos los **mismos nombres de la clase**: `mover_derecha` , `mover_izquierda` , `mover_arriba` , `mover_abajo` . El nombre lo elegís vos; solo tiene que **coincidir** con el que uses en el código.

5. **Ya que estás**, creá dos acciones más (las usamos en las Partes 5 y 6):

ACCIÓN	TECLA SUGERIDA
<code>correr</code>	Shift
<code>saltar</code>	Espacio

💡 **Por qué así:** en el código vamos a preguntar `Input.is_action_pressed("mover_derecha")` . El código **no sabe ni le importa** qué tecla es: solo pregunta por la **acción**. Cambiar el control es cambiar el mapeo, no el código.

✅ **Punto de control 2:** tenés 6 acciones en el Input Map (4 de movimiento + `correr` + `saltar`).

🚫 **No me toma la tecla / no aparece la acción**

▼ **Abrí para ver soluciones**

- Después de escribir el nombre hay que clic en **Add**; si no, la acción no se crea.
- La tecla se agrega con el `+` de esa fila, y hay que **confirmar con OK**.
- Los nombres van **sin espacios ni mayúsculas** y **exactamente igual** que en el código (`mover_derecha` , no `Mover Derecha`).

📦 **Parte 3 — Movimiento en 4 direcciones (con `delta`)**

Conceptos: `_process(delta)` , `Vector2` para la dirección, y la **regla de oro del `delta`** .

Reemplazá el script por esto:

```

extends CharacterBody2D

var nombre = "Explorador"
var velocidad_normal = 200
var velocidad_rapida = 400
var velocidad = velocidad_normal

func _ready():
    print("=== " + nombre + " listo ===")
    print("Movete con WASD o las flechas")

func _process(delta):
    # 1) Armamos la dirección según las teclas apretadas
    var direccion = Vector2.ZERO
    if Input.is_action_pressed("mover_derecha"):
        direccion.x += 1
    if Input.is_action_pressed("mover_izquierda"):
        direccion.x -= 1
    if Input.is_action_pressed("mover_abajo"):
        direccion.y += 1
    if Input.is_action_pressed("mover_arriba"):
        direccion.y -= 1

    # 2) Movemos, SIEMPRE multiplicando por delta
    position += direccion * velocidad * delta

```

Apretá **F6** y movete con el teclado. **Clic en la ventana del juego** para que reciba las teclas.

 **La regla de oro del `delta`:** `position += direccion * velocidad * delta`. Como `velocidad` es **200 px por segundo**, al multiplicar por `delta` (el tiempo del frame) el personaje avanza lo mismo **por segundo** en cualquier máquina — a 60 o a 30 FPS. Sin `delta`, en una compu más rápida iría más rápido. **Todo movimiento en `_process()` se multiplica por `delta`. Siempre.**

 **El eje Y está invertido:** en Godot, **Y positivo es hacia abajo**. Por eso "arriba" es `direccion.y -= 1` y "abajo" es `direccion.y += 1`. Es la fuente de confusión más común.

 **Probá vos:** cambiá `velocidad_normal` de `200` a `400`. ¿Se mueve más rápido? Vuelvo a `200`.

 **Punto de control 3:** el personaje se mueve en las 4 direcciones.

No se mueve

▼ Abrí para ver soluciones

- ¿Hiciste **clic en la ventana del juego**? Sin foco, no llegan las teclas.
- Los nombres de las acciones tienen que ser **idénticos** a los del Input Map.
- El movimiento va en `_process(delta)`, no en `_ready()`.
- ¿Te olvidaste el `* delta`? Sin él anda "raro" o rapidísimo, pero debería moverse igual — si no se mueve, es el foco o los nombres.

Parte 4 — Límites de pantalla con `clamp()`

Concepto: usar `clamp()` para que el personaje no se escape de la ventana.

`clamp(valor, minimo, maximo)` **recorta** un número para que quede dentro de un rango: si se pasa del máximo, lo deja en el máximo; si baja del mínimo, en el mínimo.

Agregá esto **al final** de `_process(delta)` (después de mover):

```
# 3) No dejar que se vaya de la pantalla
var limites = get_viewport_rect().size
position.x = clamp(position.x, 0, limites.x)
position.y = clamp(position.y, 0, limites.y)
```

Apretá **F6** e intentá salir por los bordes: el personaje **se frena** contra el borde.

 `get_viewport_rect().size` te da el **tamaño de la ventana** (ancho y alto). Con `clamp(position.x, 0, ancho)` obligamos a que la X esté siempre entre `0` y el ancho. Igual la Y. Así el personaje queda **encerrado** en la pantalla.

 **Probá vos:** comentá las dos líneas del `clamp` (poné `#` adelante) y fijate cómo ahora **sí** se escapa. Descomentalas de nuevo.

✅ **Punto de control 4:** el personaje no puede salir de la ventana.



Parte 5 — Correr (`just_pressed` / `just_released`)

Concepto: cambiar la velocidad con `is_action_just_pressed` y `is_action_just_released` .

Queremos: **mientras** mantengas **Shift**, corre (más rápido); al **soltar**, vuelve a caminar. Agregá esto al **principio** de `_process(delta)` (antes de armar la dirección):

```
# Correr: cambia la velocidad al apretar y al soltar
if Input.is_action_just_pressed("correr"):
    velocidad = velocidad_rapida
    print("🏃 Corriendo")
if Input.is_action_just_released("correr"):
    velocidad = velocidad_normal
    print("🚶 Caminando")
```

Apretá **F6**, movete y mantené **Shift**: se mueve más rápido y aparece "🏃 Corriendo" **una vez**. Al soltar, "🚶 Caminando".

💡 Las tres variantes de input (de la clase):

- `is_action_pressed` → **mientras** esté apretada (lo usamos para moverse).
- `is_action_just_pressed` → **el instante** en que se aprieta (empezar a correr).
- `is_action_just_released` → **el instante** en que se suelta (dejar de correr).

Fijate que los mensajes aparecen **una sola vez** por pulsación, no 60 veces por segundo: por eso usamos las variantes `just_` .

✅ **Punto de control 5:** con Shift corre más rápido; al soltar, vuelve a la velocidad normal.



Parte 6 — Dash / salto (`just_pressed`)

Concepto: una acción que se dispara **una sola vez** por pulsación con `is_action_just_pressed` .

En un juego **cenital** (visto desde arriba) no hay "salto" con gravedad, así que representamos el salto como un **dash**: un **impulso** que te lanza un toque en la dirección en la que venías. Lo importante es el concepto: se dispara **una vez** por cada vez que apretás.

Necesitamos **recordar la última dirección**. Agregá esta variable arriba, junto a las otras:

```
var distancia_dash = 120
var ultima_direccion = Vector2(1, 0) # arranca mirando a la derecha
```

Dentro de `_process(delta)`, **justo después** de armar `direccion` (y antes del `position += ...`), guardá la última dirección real:

```
# Recordar hacia dónde nos movemos (para el dash)
if direccion != Vector2.ZERO:
    ultima_direccion = direccion
```

Y agregá el dash **después** de mover (antes del `clamp`):

```
# Dash: un impulso que se dispara UNA vez por pulsación
if Input.is_action_just_pressed("saltar"):
    position += ultima_direccion * distancia_dash
    print("💨 ¡Dash!")
```

Apretá **F6**: movete en alguna dirección y tocá **Espacio** — el personaje **pega un salto** en esa dirección, y "💨 ¡Dash!" aparece **una sola vez** por cada toque.

💡 **Por qué `just_pressed` y no `pressed`**: si usáramos `is_action_pressed`, el dash se dispararía **60 veces por segundo** mientras mantenés Espacio (el personaje volaría sin control). Con `just_pressed`, **un toque = un dash**. Esa es la diferencia clave.

✅ **Punto de control 6**: cada toque de Espacio hace un dash en la dirección actual.

🏆 Parte 7 — Todo junto

Este es el script final completo. Revisalo entendiendo **qué parte hace qué** (marcado con comentarios):

```

extends CharacterBody2D

# ---- Variables (Parte 1) ----
var nombre = "Explorador"
var velocidad_normal = 200
var velocidad_rapida = 400
var velocidad = velocidad_normal
var distancia_dash = 120
var ultima_direccion = Vector2(1, 0)

func _ready():
    print("=== " + nombre + " listo ===")
    print("WASD/flechas: mover | Shift: correr | Espacio: dash")

func _process(delta):
    # --- Correr: cambia la velocidad (Parte 5) ---
    if Input.is_action_just_pressed("correr"):
        velocidad = velocidad_rapida
        print("🏃 Corriendo")
    if Input.is_action_just_released("correr"):
        velocidad = velocidad_normal
        print("🚶 Caminando")

    # --- Dirección en 4 direcciones (Parte 3) ---
    var direccion = Vector2.ZERO
    if Input.is_action_pressed("mover_derecha"):
        direccion.x += 1
    if Input.is_action_pressed("mover_izquierda"):
        direccion.x -= 1
    if Input.is_action_pressed("mover_abajo"):
        direccion.y += 1
    if Input.is_action_pressed("mover_arriba"):
        direccion.y -= 1

    # Recordar la última dirección real (para el dash)
    if direccion != Vector2.ZERO:
        ultima_direccion = direccion

    # --- Mover, SIEMPRE con delta (Parte 3) ---
    position += direccion * velocidad * delta

    # --- Dash: una vez por pulsación (Parte 6) ---
    if Input.is_action_just_pressed("saltar"):
        position += ultima_direccion * distancia_dash

```

```
print("💨 ¡Dash!")
```

```
# --- Límites de pantalla con clamp (Parte 4) ---  
var limites = get_viewport_rect().size  
position.x = clamp(position.x, 0, limites.x)  
position.y = clamp(position.y, 0, limites.y)
```

Cómo se juega: **WASD** o **flechas** para moverte, **Shift** para correr, **Espacio** para el dash. No podés salir de la pantalla.

✅ **Punto de control 7 (final):** todo funciona junto — mover, correr, dashear y quedar dentro de la ventana.

🔧 Errores comunes al armar el final

▼ Abrí para ver soluciones

- **Invalid call ... clamp**: `clamp` va con **tres** argumentos: `clamp(position.x, 0, limites.x)`.
- **El dash no anda**: revisá que la acción se llame `saltar` en el Input Map y que `ultima_direccion` esté declarada arriba.
- **Corre para siempre**: usaste `is_action_pressed` en vez de `is_action_just_pressed` / `just_released` para `correr`.
- **Indentación mezclada**: usá espacios o tabs en todo el archivo, no los combines.
- **Se mueve rarísimo de rápido**: te faltó el `* delta` en el `position +=`.

📁 Entrega

Entregá **una** de estas opciones (según indique el/la docente):

1. El archivo `jugador.gd`, o
2. La **carpeta del proyecto** comprimida en `.zip` (sin la carpeta `.godot/`), o
3. Un **video corto** (o GIF) mostrando el personaje moverse, correr y dashear.

Nombre del archivo: `tp3-movimiento-ApellidoNombre.zip` (o `.gd`).

✅ Checklist de autoevaluación

- ☐ Tenés un `CharacterBody2D` con `Sprite2D` y `CollisionShape2D`, y un script adjunto.
- ☐ Creaste **6 acciones** en el Input Map (4 de movimiento + `correr` + `saltar`).
- ☐ Imprimís la **configuración** del personaje en `_ready()`.

- ☐ El personaje se **mueve en 4 direcciones**.
- ☐ Todo el movimiento usa `* delta`.
- ☐ Con `correr` cambia la velocidad usando `just_pressed` y `just_released`.
- ☐ El **dash** usa `is_action_just_pressed` (una vez por pulsación).
- ☐ El personaje **no sale** de la pantalla (`clamp`).

☀ Extra (opcional, para los que quieran más)

- **Diagonales justas:** al moverte en diagonal, ahora vas **un poco más rápido** (se suman X e Y). Se arregla normalizando la dirección:

```
position += direccion.normalized() * velocidad * delta
```

`.normalized()` deja el largo del vector en 1, así la diagonal no es más veloz.

- **Margen en los límites:** el `clamp` deja que el borde del sprite se asome. Restá un margen: `clamp(position.x, 40, limites.x - 40)`.
- **Imprimir la posición solo cuando cambia:** guardá la posición anterior en una variable y compará con un `if` antes de `print()`.
- **Enfriamiento del dash (cooldown):** que no se pueda dashear de nuevo hasta que pase un ratito. Pista: una variable de tiempo que baje con `delta`.
- **De cara a la clase de física:** este mismo personaje se va a mover con `velocity` + `move_and_slide()` para que **choque** con paredes. Misma idea, motor de física real.

Recursos

- Configurar el Input Map (capturas de este TP): [docs.godotengine.org/es/4.x — Coding the player](https://docs.godotengine.org/es/4.x/Coding%20the%20player)
- Ejemplos de input: [Input examples](#)
- Usar CharacterBody2D: [Using CharacterBody2D](#)

Las capturas de este documento provienen de la **documentación oficial de Godot Engine** (Juan Linietsky, Ariel Manzur y la comunidad), bajo licencia **CC BY 4.0**.